

Exercício 1

Dê as definições formais dos seguintes modelos de planejamento probabilístico:

- a) MDP de horizonte finito
- b) MDP de horizonte infinito e descontado
- c) SSP

Resposta:

a)

No modelo de horizonte finito, a tomada de decisão do agente é delimitada por um número fixo e conhecido de passos de tempo N (também chamado de horizonte H).

Um MDP de Horizonte Finito é definido formalmente pela tupla:

$$\mathcal{M} = \langle S, A, P, R, N \rangle$$

onde:

$N \in \mathbb{N}$ é o horizonte temporal máximo do processo.

O objetivo é encontrar uma política que maximize o valor esperado da soma total de recompensas acumuladas ao longo dos N passos:

$$U([s_0, s_1, \dots, s_N]) = \mathbb{E} \left[\sum_{t=0}^{N-1} R(s_t, a_t, s_{t+1}) \right]$$

Propriedades Notáveis:

- Não Estacionariedade da Política Ótima: A ação ótima a ser tomada depende explicitamente do tempo restante t . Por exemplo, a melhor escolha no passo $t = 1$ (com $N - 1$ passos restantes) pode ser diferente da melhor escolha no passo $t = N - 1$ (com apenas 1 passo restante). Portanto, a política ótima é dependente do tempo: $\pi_t^*(s)$.

b)

No modelo de horizonte infinito, o processo continua indefinidamente ($N \rightarrow \infty$). Para evitar somas infinitas que não convergem e para modelar a preferência por recompensas imediatas em vez de futuras, introduz-se um fator de desconto.

Um MDP de Horizonte Infinito e Descontado é definido pela tupla:

$$\mathcal{M} = \langle S, A, P, R, \gamma \rangle$$

onde:

$\gamma \in [0, 1)$ é o fator de desconto.

O objetivo do agente é maximizar o valor esperado da soma de recompensas futuras descontadas:

$$U([s_0, s_1, s_2, \dots]) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

Propriedades Notáveis:

- **Convergência Matemáticas:** Como R é limitado e $\gamma < 1$, a série geométrica garante que a utilidade permaneça finita: $\sum_{t=0}^{\infty} \gamma^t R_{max} = \frac{R_{max}}{1-\gamma}$.
- **Estacionariedade da Política Ótima:** O futuro restante a partir de qualquer estado s é sempre infinito. Portanto, o tempo absoluto não altera a escolha ótima. A política ótima $\pi^*(s)$ é estacionária (mapeia o estado s diretamente para a ação a , independentemente do instante t).

c)

O modelo de Caminho Estocástico mais Curtos (SSP) generaliza a busca do caminho mais curto para ambientes probabilísticos. O horizonte não é fixado a priori (como no caso finito) nem infinito, mas indefinido: o processo termina no momento em que o agente atinge um estado objetivo.

Um SSP é tipicamente formalizado pela tupla:

$$\mathcal{M} = \langle S, A, P, C, s_0, G \rangle$$

onde:

$G \subset S$ é um conjunto não vazio de estados objetivos (goal states), que são absorventes e possuem custo de saída zero: $P(g | g, a) = 1$ e $C(g, a) = 0$ para todo $g \in G$ e $a \in A$.

$C(s, a) > 0$ é a função de custo para a execução de uma ação a em um estado $s \notin G$ (costuma-se usar custos estritamente positivos em vez de recompensas para penalizar passos adicionais).

$s_0 \in S$ é o estado inicial.

O objetivo é encontrar uma política que minimize o custo esperado acumulado até alcançar um estado de objetivo $g \in G$:

$$J^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{T-1} C(s_t, a_t) \mid s_0 = s \right]$$

onde T é uma variável aleatória que representa o instante de parada (primeira vez que $s_T \in G$).

Para garantir a existência de uma solução bem-definida e finita no SSP, a teoria exige duas premissas essenciais:

- **Existência de Política Própria (Proper Policy):** Existe pelo menos uma política que alcança o objetivo G a partir de qualquer estado inicial com probabilidade 1.
- **Penalização de Políticas Impróprias:** Toda política imprópria (uma política que corre o risco de nunca atingir G) incorre em um custo esperado infinito para os estados a partir dos quais não garante a chegada ao objetivo.

Exercício 2

Dado um MDP de horizonte infinito e descontado, dê as definições para as seguintes funções utilidade:

- a) função valor de estado ($V(s)$).
- b) função valor de estado e ação ($Q(s,a)$).

Resposta:

Um Processo de Decisão de Markov (MDP) com horizonte infinito e fator de desconto trata da tomada de decisão em ambientes estocásticos onde a sequência de ações não possui um término fixo e as recompensas futuras têm menos peso do que as imediatas.

O Fator de Desconto (γ):

Antes de definir as funções de valor, vale destacar a utilidade aditiva descontada. Se o agente recebe uma sequência de recompensas $R_t, R_{t+1}, R_{t+2}, \dots$, o retorno total acumulado a partir do tempo t é:

$$U([R_t, R_{t+1}, R_{t+2}, \dots]) = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

Onde $\gamma \in [0, 1)$ é o fator de desconto.

Ele garante que a soma infinita convirja para um valor finito e reflete a preferência por recompensas mais imediatas.

a)

A função valor de estado $V^\pi(s)$ mede o retorno esperado descontado que o agente obterá ao começar no estado s e, a partir dele, seguir a política π .

Matematicamente, é expressa como o valor esperado do retorno acumulado:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k} \mid S_t = s \right]$$

A utilidade de um estado s sob uma política π pode ser decomposta recursivamente usando a Equação de Bellman:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s' \mid s, \pi(s)) V^\pi(s')$$

$R(s, a)$: É a recompensa imediata obtida ao executar a ação a no estado s .

$P(s' \mid s, a)$: É a probabilidade de transição para o estado s' , dado que a ação a foi tomada em s .

$\gamma \sum_{s'} P(s' \mid s, \pi(s)) V^\pi(s')$: É a soma ponderada das utilidades esperadas de todos os possíveis próximos estados s' , descontadas por γ .

A função valor ótima $V^*(s)$ representa o maior retorno esperado que se pode obter a partir do estado s , escolhendo a melhor ação possível:

$$V^*(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right]$$

b)

A função valor de estado-ação $Q^\pi(s, a)$ mede o retorno esperado descontado que o agente obterá ao começar no estado s , executar a ação a no primeiro passo e, a partir de então, seguir a política π .

Matematicamente:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k} \mid S_t = s, A_t = a \right]$$

A diferença fundamental em relação a $V(s)$ é que $Q(s, a)$ fixa a primeira ação como a , sem impor que a venha da política $\pi(s)$.

A decomposição recursiva de Bellman para a função Q é expressa por:

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) Q^\pi(s', \pi(s'))$$

No caso da política ótima π^* , a função $Q^*(s, a)$ representa o retorno máximo ao escolher a em s e agir otimamente a partir do próximo estado:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a')$$

extra

Relação entre $V(s)$ e $Q(s, a)$

As duas funções estão intimamente ligadas:

- Obter $V(s)$ a partir de $Q(s, a)$ sob a política ótima:

$$V^*(s) = \max_a Q^*(s, a)$$

(O valor do estado é simplesmente o valor Q da melhor ação disponível nele).

- Obter $Q(s, a)$ a partir de $V(s)$:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')$$

- Política Ótima $\pi^*(s)$: A política ótima pode ser extraída diretamente de $Q^*(s, a)$ sem a necessidade de conhecer o modelo de transição $P(s' | s, a)$:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Exercício 3

Dada uma política p para um MDP de horizonte infinito descontado, qual é a equação que define a avaliação da política p ?

Resposta:

O processo de determinar a função valor $V^\pi(s)$ para uma política fixa π é chamado de avaliação de política.

Para um MDP de horizonte infinito com fator de desconto $\gamma \in [0, 1)$, a equação que define a avaliação de uma política π é a Equação de Expectativa de Bellman:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s' | s, \pi(s)) V^\pi(s') \quad \forall s \in S$$

Onde:

- $V^\pi(s)$: O valor (utilidade esperada descontada) de estar no estado s e seguir a política π .
- $R(s, \pi(s))$: A recompensa imediata recebida por executar a ação prescrita pela política $\pi(s)$ no estado s .
- γ : O fator de desconto ($0 \leq \gamma < 1$), que garante a convergência no horizonte infinito.
- $P(s' | s, \pi(s))$: A probabilidade de transição do estado s para o estado s' ao executar a ação $\pi(s)$.
- $\sum_{s'} P(s' | s, \pi(s)) V^\pi(s')$: O valor esperado do próximo estado s' .

Como a política π é fixa, não há o operador de maximização ($\max_a$) na equação. Isso significa que, se houver N estados no espaço de estados S :

- A equação acima gera um sistema linear de N equações a N incógnitas (uma incógnita $V^\pi(s)$ para cada estado s).
- Esse sistema pode ser resolvido exatamente usando álgebra linear com complexidade de tempo $O(N^3)$, na forma matricial:

$$\mathbf{V}^\pi = \mathbf{R}^\pi + \gamma \mathbf{P}^\pi \mathbf{V}^\pi \implies \mathbf{V}^\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{R}^\pi$$

- Para ambientes com muitos estados (onde inverter a matriz é muito custoso), usa-se a resolução iterativa usando a avaliação iterativa de política, onde os valores são atualizados sucessivamente até a convergência:

$$V_{k+1}(s) \leftarrow R(s, \pi(s)) + \gamma \sum_{s'} P(s' | s, \pi(s)) V_k(s')$$

Exercício 4

Descreva a Equação de Otimalidade da Bellman para MDPs de horizonte infinito descontado.

Resposta:

A Equação de Otimalidade de Bellman é o pilar fundamental para caracterizar o comportamento de um agente racional em um Processo de Decisão de Markov (MDP) de horizonte infinito descontado.

Ela estabelece a relação de consistência que a função valor ótima $V^*(s)$ — ou a função Q ótima $Q^*(s, a)$ — deve satisfazer quando o agente escolhe a melhor ação possível em todos os passos.

A função valor ótima $V^*(s)$ representa o retorno total descontado esperado ao começar no estado s e agir otimamente a partir de então.

$$V^*(s) = \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right] \quad \forall s \in S$$

Onde:

- $V^*(s)$: A utilidade máxima esperada de estar no estado s .
- $\max_{a \in A(s)}$: O operador de seleção que busca a ação a disponível no estado s que maximiza o retorno total esperado.
- $R(s, a)$: A recompensa imediata obtida ao tomar a ação a no estado s .
- γ : O fator de desconto ($0 \leq \gamma < 1$), que assegura que a soma infinita convirja para um valor finito e prioriza recompensas mais próximas.
- $P(s' | s, a)$: A probabilidade de transitar para o estado s' após executar a ação a no estado s .
- $\sum_{s'} P(s' | s, a) V^*(s')$: A utilidade média esperada do próximo estado s' , ponderada pelas probabilidades do ambiente.

A essência da equação é o Princípio da Otimalidade de Bellman:

A utilidade ótima de um estado s é igual à recompensa imediata máxima recebida pela melhor ação a , somada ao valor ótimo descontado dos estados subsequentes s' resultantes dessa ação.

Se expressarmos a otimalidade em termos do valor da dupla estado-ação (Q^*), a equação toma a seguinte forma:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a')$$

Neste caso, a otimalidade no próximo estado s' é garantida pela escolha do maior valor entre todas as ações a' disponíveis em s' ($\max_{a'} Q^*(s', a')$).

I. Não Linearidade

Ao contrário da equação de avaliação de política (que é um sistema de equações lineares), a equação de otimalidade contém o operador max, o que a torna não linear. Por esse motivo, não é possível resolvê-la diretamente com métodos de álgebra linear tradicional

(como a inversão de matrizes).

II. Existência e Unicidade da Solução

Para qualquer MDP descontado de horizonte infinito com $0 \leq \gamma < 1$, existe uma única solução V^* para o sistema de equações de Bellman. A aplicação iterativa do operador de otimalidade forma uma contração (Contraction Mapping), garantindo que os métodos convergiram para o mesmo valor fixo.

III. Extração da Política Ótima (π^*)

Uma vez calculados os valores de $V^*(s)$ para todos os estados, a política ótima π^* é obtida escolhendo, em cada estado s , a ação que atinge o máximo na equação:

$$\pi^*(s) = \arg \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right]$$

Se a função $Q^*(s, a)$ for conhecida, a escolha simplifica-se para:

$$\pi^*(s) = \arg \max_{a \in A(s)} Q^*(s, a)$$

Dois algoritmos clássicos são apresentados para resolver a Equação de Otimalidade de Bellman:

1) Iteração de Valor:

Aplica a equação de otimalidade como uma regra de atualização iterativa até que os valores de $V(s)$ convirjam:

$$V_{k+1}(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_k(s') \right]$$

2) Iteração de Política:

Alterna entre avaliar uma política fixa (sistema linear) e melhorá-la de forma gulosa com base nos valores calculados.

Exercício 5

Dê as definições para os seguintes tipos de política:

- a) política parcial
- b) política própria
- c) política gulosa
- d) política ótima

Resposta:

Uma política π é uma regra de decisão que especifica qual ação o agente deve tomar em cada estado do ambiente. Formamente, é uma função ou mapeamento $\pi : S \rightarrow A$.

a)

Uma política parcial é uma função que não define ações para todos os estados possíveis do MDP, estando especificada apenas para um subconjunto do espaço de estados $S_{definido} \subset S$.

Em ambientes muito grandes, na busca em árvore online ou no uso de algoritmos de planejamento parcial (como em Real-Time Dynamic Programming - RTDP), calcula-se a tomada de decisão apenas para os estados que o agente é capaz de alcançar a partir do seu estado atual dentro de certo horizonte, deixando os estados irrelevantes ou inacessíveis sem uma ação mapeada.

Em contraste, uma política completa especifica uma ação válida $\pi(s)$ para todos os estados $s \in S$ que o agente possa eventualmente visitar.

b)

Uma política própria é uma política que garante atingir um estado terminal (ou estado meta) com probabilidade 1, independentemente de qual seja o estado inicial do agente.

Essa definição aplica-se especificamente a MDPs que contêm estados de término/meta — um modelo frequentemente tratado como o problema do Caminho Mínimo Estocástico (SSP).

Uma política π é dita própria se, a partir de qualquer estado $s \in S$, o limite da probabilidade de o agente alcançar o estado meta $s_g \in G$ à medida que o tempo $t \rightarrow \infty$ é igual a 1:

$$\lim_{t \rightarrow \infty} P(S_t \in G \mid S_0 = s, \pi) = 1$$

Política Imprópria: Se existir ao menos um estado inicial a partir do qual a política corra o risco de prender o agente em um loop infinito sem nunca alcançar o estado terminal, essa política é classificada como imprópria.

c)

Uma política gulosa é aquela que seleciona, em cada estado s , a ação que maximiza a utilidade esperada de curto prazo em relação a uma função valor atual/arbitrária $V(s)$ ou $Q(s, a)$.

Formulação com Função Valor de Estado $V(s)$:

$$\pi_{gulosa}(s) = \arg \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V(s') \right]$$

Formulação com Função Valor de Estado-Ação $Q(s, a)$:

$$\pi_{gulosa}(s) = \arg \max_{a \in A(s)} Q(s, a)$$

O conceito de política gulosa é central no algoritmo de Iteração de Política. A cada passo de melhoria da política, o agente deriva uma nova política gulosa a partir da função valor calculada na etapa de avaliação da política anterior.

d)

A política ótima, denotada por π^* , é a política que maximiza o retorno total descontado esperado a partir de qualquer estado inicial $s \in S$.

Uma política π^* é ótima se a sua função valor $V^{\pi^*}(s)$ for estritamente superior ou igual à função valor $V^\pi(s)$ de qualquer outra política π , para todos os estados $s \in S$:

$$V^{\pi^*}(s) \geq V^\pi(s) \quad \forall \pi, \forall s \in S$$

Uma política ótima é obtida agindo de forma gulosa em relação à função valor ótima $V^*(s)$:

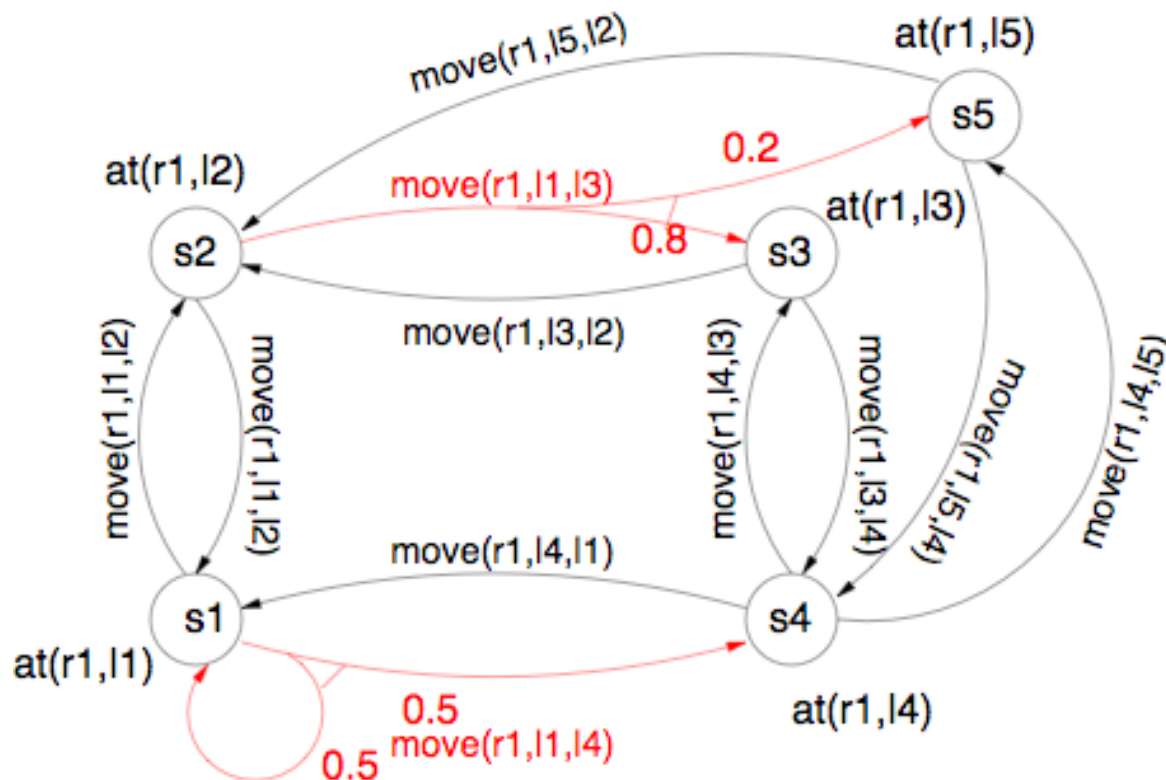
$$\pi^*(s) = \arg \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right]$$

Unicidade e Estacionariedade no Horizonte Infinito:

Demonstra-se que, para MDPs descontados de horizonte infinito, sempre existe pelo menos uma política ótima que é estática/estacionária e determinística — ou seja, a ação ideal depende apenas do estado atual s , e não do tempo decorrido ou do histórico do agente.

Exercício 6

Considere o MDP ilustrado da figura abaixo.



Considere a função utilidade local como $R(s) - C(s,a)$ (isto é, recompensa menos o custo); sendo que definimos a função custo para os pares estado/ação, isto é $C(s,a)$, como:

- $C(s,wait) = 0$ para todo estado s ,
- $C(s,a) = 1$ para toda ação “horizontal” ,
- $C(s,a) = 100$ para toda ação “vertical” ,

e a função recompensa para o estado, sendo $R(s) = 0$ para todo estado s , exceto para os estados s_4 e s_5 que são definidas por:

- $R(s_4) = 100$
- $R(s_5) = -100$

Considere também que é dada a política p especificada conforme a tabela abaixo.

Estado s	Ação $\pi(s)$
s_1	$move(r1, l1, l2)$
s_2	$move(r1, l2, l3)$
s_3	$move(r1, l3, l4)$
s_4	wait
s_5	$move(r1, l5, l4)$

Resposta:

Horizonte e Desconto:

Assumimos $\gamma = 1$ (ou sem desconto explícito indicado para as iterações).

Recompensas e Custos:

- $R(s) - C(s, a)$: $R(s_4) = 100$, $R(s_5) = -100$, e $R(s_1) = R(s_2) = R(s_3) = 0$.
- Custo de ações: $C(wait) = 0$, $C(horizontal) = 1$, $C(vertical) = 100$.
- Recompensa Líquida $R'(s, a) = R(s) - C(s, a)$:
 - $R'(s_1, move(r1, l1, l2)) = 0 - 1 = -1$
 - $R'(s_2, move(r1, l2, l3)) = 0 - 1 = -1$
 - $R'(s_3, move(r1, l3, l4)) = 0 - 1 = -1$
 - $R'(s_4, wait) = 100 - 0 = 100$
 - $R'(s_5, move(r1, l5, l4)) = -100 - 1 = -101$

Probabilidades de Transição dada a política π :

- Em s_1 : $\pi(s_1) = move(r1, l1, l2)$ leva a s_2 com $P = 0.8$ e a s_3 com $P = 0.2$.
- Em s_2 : $\pi(s_2) = move(r1, l2, l3)$ leva a s_3 com $P = 1.0$.
- Em s_3 : $\pi(s_3) = move(r1, l3, l4)$ leva a s_4 com $P = 0.5$ e s_1 com $P = 0.5$.
- Em s_4 : $\pi(s_4) = wait$ permanece em s_4 com $P = 1.0$.
- Em s_5 : $\pi(s_5) = move(r1, l5, l4)$ leva a s_4 com $P = 1.0$.

Iterações da Avaliação de Política ($V_0(s) = 0$ para todo s):

- 1ª Iteração ($k = 1$):
 - $V^1(s_1) = -1 + [0.8(0) + 0.2(0)] = -1$
 - $V^1(s_2) = -1 + 1.0(0) = -1$
 - $V^1(s_3) = -1 + [0.5(0) + 0.5(0)] = -1$
 - $V^1(s_4) = 100 + 1.0(0) = 100$
 - $V^1(s_5) = -101 + 1.0(0) = -101$
- 2ª Iteração ($k = 2$):
 - $V^2(s_1) = -1 + [0.8(-1) + 0.2(-1)] = -1 - 1 = -2$
 - $V^2(s_2) = -1 + 1.0(-1) = -2$
 - $V^2(s_3) = -1 + [0.5(100) + 0.5(-1)] = -1 + 49.5 = 48.5$
 - $V^2(s_4) = 100 + 1.0(100) = 200$
 - $V^2(s_5) = -101 + 1.0(100) = -1$
- 3ª Iteração ($k = 3$):
 - $V^3(s_1) = -1 + [0.8(-2) + 0.2(48.5)] = -1 + [-1.6 + 9.7] = 7.1$
 - $V^3(s_2) = -1 + 1.0(48.5) = 47.5$
 - $V^3(s_3) = -1 + [0.5(200) + 0.5(-2)] = -1 + [100 - 1] = 98$
 - $V^3(s_4) = 100 + 1.0(200) = 300$
 - $V^3(s_5) = -101 + 1.0(200) = 99$

Estado s	V^π (1ª iteração)	V^π (2ª iteração)	V^π (3ª iteração)
s_1	-1	-2	7,1
s_2	-1	-2	47,5
s_3	-1	48,5	98
s_4	100	200	300
s_5	-101	-1	99

Exercício 7

Considere um cenário onde um robô vigilante deve permanecer num corredor de um prédio, sem atrapalhar a passagem das pessoas. Uma vez que do lado esquerdo desse corredor existem portas, o robô atrapalha menos quando permanece do lado direito. Além disso, quando o robô se desloca, a posição desejada é alcançada com probabilidade 0.7 (e com probabilidade 0.3, ele continua na mesma posição em que estava). Em cada instante, o robô deve decidir qual a melhor ação: permanecer onde está ou se mover para direita ou esquerda. Este problema pode ser modelado por um MDP $\langle S, A, P, R, C \rangle$ cujo grafo de transição de estados é dado pela Figura 4. As arestas são rotuladas pelas ações *permanecer*, *paraesq* e *paradir*. E suas probabilidades de transição. Ovais são rotulados pelos estados e o valor de recompensa.

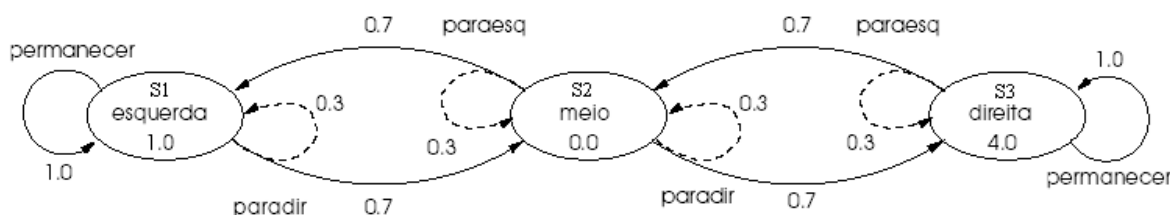


Figura 4. Diagrama de transição de estados para o robô vigilante

Notem que existem três possíveis estados no conjunto S :

- s1: Loc = esquerda
- s2: Loc = meio
- s3: Loc = direita

O conjunto de ações $A = \{\textit{permanecer}, \textit{paraesq}, \textit{paradir}\}$ especifica as possíveis ações do robô; P é a função de transição de estados probabilística, dada por:

- $P(s1 \mid \textit{permanecer}, s1) = 1.0$
- $P(s1 \mid \textit{paradir}, s1) = 0.3$
- $P(s2 \mid \textit{paradir}, s1) = 0.7$
- $P(s2 \mid \textit{paradir}, s2) = 0.3$
- $P(s3 \mid \textit{paradir}, s2) = 0.7$
- $P(s2 \mid \textit{paraesq}, s2) = 0.3$
- $P(s1 \mid \textit{paraesq}, s2) = 0.7$
- $P(s3 \mid \textit{permanecer}, s3) = 1$
- $P(s3 \mid \textit{paraesq}, s3) = 0.3$
- $P(s2 \mid \textit{paraesq}, s3) = 0.7$

R é a função que associa recompensas a estados (de modo que posições preferidas sejam melhor recompensadas):

- $R(s1) = 1.0$
- $R(s2) = 0.0$
- $R(s3) = 4.0$

e, finalmente, C é a função custo das ações do robô (de modo que ações preferidas tenham custo menor):

- $C(\textit{permanecer}) = 0$

- $C(\text{paraesq})=1$
- $C(\text{paradir})=1$

Considere a política inicial:

Estado	Ação
s_1	<i>permanecer</i>
s_2	<i>paradir</i>
s_3	<i>permanecer</i>

Sabendo que neste MDP queremos maximizar o valor esperado acumulado descontado de R-C, calcule (manualmente) a política ótima estacionária, com $\gamma = 0.9$, usando o algoritmo de iteração de política especificado a seguir. Note que o algoritmo usa a ideia mas simples de resolução de sistema de equações lineares (Aula 17, slide 58).

Algorithm 1: Iteração-de-Política(S, A, P, R, C, γ)

```

1  $\pi \leftarrow \emptyset$  ;
2 Selecione qualquer  $\pi' \neq \emptyset$  ; // Política inicial dada
3 while  $\pi' \neq \pi$  do
4    $\pi \leftarrow \pi'$  ;
5   foreach  $s \in S$  do
6      $E_\pi(s) \leftarrow$  solução do sistema de equações: ;
7      $E_\pi(s) = R(s) - C(\pi(s)) + \gamma \sum_{s' \in S} P(s' | s, \pi(s)) E_\pi(s')$  ;
8   foreach  $s \in S$  do
9     if  $\exists a \in A \mid E_\pi(s) < R(s) - C(a) + \gamma \sum_{s' \in S} P(s' | s, a) E_\pi(s')$  then
10       $\pi'(s) \leftarrow a$  ;
11     else
12       $\pi'(s) \leftarrow \pi(s)$  ;
13 return  $\pi$  ;
```

Resposta:

- Conjunto de Estados: $S = \{s_1, s_2, s_3\}$
- Conjunto de Ações: $A = \{\text{permanecer}, \text{paraesq}, \text{paradir}\}$
- Fator de Desconto: $\gamma = 0.9$
- Recompensas dos Estados $R(s)$:

$$R(s_1) = 1.0, \quad R(s_2) = 0.0, \quad R(s_3) = 4.0$$

- Custos das Ações $C(a)$:

$$C(\text{permanecer}) = 0, \quad C(\text{paraesq}) = 1, \quad C(\text{paradir}) = 1$$

- Política Inicial (π_0):

$$\pi_0(s_1) = \text{permanecer}, \quad \pi_0(s_2) = \text{paradir}, \quad \pi_0(s_3) = \text{permanecer}$$

1ª Iteração: Avaliação da Política (π_0)

A equação de avaliação de política é dada por:

$$E_\pi(s) = R(s) - C(\pi(s)) + \gamma \sum_{s' \in S} P(s' | s, \pi(s)) E_\pi(s')$$

1. **Para** s_1 com $\pi_0(s_1) = \text{permanecer}$ ($C = 0, P(s_1 | s_1) = 1.0$):

$$E_\pi(s_1) = 1.0 - 0 + 0.9(1.0 \cdot E_\pi(s_1)) \implies 0.1E_\pi(s_1) = 1.0 \implies \mathbf{E_\pi(s_1) = 10}$$

2. **Para** s_3 com $\pi_0(s_3) = \text{permanecer}$ ($C = 0, P(s_3 | s_3) = 1.0$):

$$E_\pi(s_3) = 4.0 - 0 + 0.9(1.0 \cdot E_\pi(s_3)) \implies 0.1E_\pi(s_3) = 4.0 \implies \mathbf{E_\pi(s_3) = 40}$$

3. **Para** s_2 com $\pi_0(s_2) = \text{paradir}$ ($C = 1, P(s_2 | s_2) = 0.3, P(s_3 | s_2) = 0.7$):

$$E_\pi(s_2) = 0.0 - 1.0 + 0.9[0.3E_\pi(s_2) + 0.7E_\pi(s_3)]$$

Substituindo $E_\pi(s_3) = 40$:

$$E_\pi(s_2) = -1.0 + 0.27E_\pi(s_2) + 0.63(40) \implies 0.73E_\pi(s_2) = 24.2 \implies \mathbf{E_\pi(s_2) \approx 33.15}$$

1ª Iteração: Melhoria da Política

Avaliamos a utilidade esperada $Q(s, a) = R(s) - C(a) + \gamma \sum_{s'} P(s' | s, a) E_\pi(s')$ para cada ação:

- **No Estado s_1 :**

$$Q(s_1, \text{permanecer}) = 1.0 - 0 + 0.9(1.0 \times 10) = \mathbf{10.0}$$

$$Q(s_1, \text{paradir}) = 1.0 - 1 + 0.9[0.3(10) + 0.7(33.15)] = 0.9(26.205) = \mathbf{23.58}$$

Como $23.58 > 10.0$, atualizamos $\pi_1(s_1) = \text{paradir}$.

- **No Estado s_2 :**

$$Q(s_2, \text{paradir}) = 0.0 - 1 + 0.9[0.3(33.15) + 0.7(40)] = \mathbf{33.15}$$

$$Q(s_2, \text{paraesq}) = 0.0 - 1 + 0.9[0.7(10) + 0.3(33.15)] = \mathbf{14.25}$$

Mantemos $\pi_1(s_2) = \text{paradir}$.

- **No Estado s_3 :**

$$Q(s_3, \text{permanecer}) = 4.0 - 0 + 0.9(1.0 \times 40) = \mathbf{40.0}$$

$$Q(s_3, \text{paraesq}) = 4.0 - 1 + 0.9[0.7(33.15) + 0.3(40)] = \mathbf{34.68}$$

Mantemos $\pi_1(s_3) = \text{permanecer}$.

A nova política obtida é $\pi_1 = \{\text{paradir}, \text{paradir}, \text{permanecer}\}$. Como $\pi_1 \neq \pi_0$, seguimos para a segunda iteração.

2ª Iteração: Avaliação da Nova Política (π_1)

1. **Para** s_3 : $\mathbf{E_\pi(s_3) = 40}$

2. **Para** s_2 : $\mathbf{E_\pi(s_2) \approx 33.15}$

3. **Para** s_1 com $\pi_1(s_1) = \text{paradir}$ ($C = 1, P(s_1 | s_1) = 0.3, P(s_2 | s_1) = 0.7$):

$$E_\pi(s_1) = 1.0 - 1.0 + 0.9[0.3E_\pi(s_1) + 0.7E_\pi(s_2)]$$

$$E_\pi(s_1) = 0.27E_\pi(s_1) + 0.63(33.15) \implies 0.73E_\pi(s_1) = 20.8845 \implies \mathbf{E_\pi(s_1) \approx 28.61}$$

2ª Iteração: Melhoria da Política

- No Estado s_1 :

$$Q(s_1, \text{permanecer}) = 1.0 - 0 + 0.9(1.0 \times 28.61) = \mathbf{26.75}$$

$$Q(s_1, \text{paradir}) = 0.0 + 0.9[0.3(28.61) + 0.7(33.15)] = \mathbf{28.61}$$

Ação *paradir* permanece como a melhor opção.

Como $\pi_2 = \pi_1$, o algoritmo convergiu.

3. Resultado Final

A política ótima estacionária π^* encontrada pelo algoritmo é:

Estado s	Ação Ótima $\pi^*(s)$
s_1	<i>paradir</i>
s_2	<i>paradir</i>
s_3	<i>permanecer</i>

Exercício 8

Explique a diferença entre os algoritmos Iteração de Valor (IV), Iteração de Política (IP) e Real Time Dynamic Programming (RTDP)

Resposta:

Os três algoritmos utilizam o princípio da Programação Dinâmica para resolver Processos de Decisão de Markov (MDPs), mas diferem na forma como atualizam as funções de valor e na maneira como exploram o espaço de estados:

Iteração de Valor:

- Mecanismo: Atualiza diretamente a função valor dos estados $V(s)$ através da aplicação iterativa da Equação de Otimalidade de Bellman.
- Funcionamento: Em cada iteração, recalcula o valor $V(s)$ para todos os estados $s \in S$ aplicando o operador de maximização sobre todas as ações possíveis:

$$V_{k+1}(s) \leftarrow \max_{a \in A} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_k(s') \right]$$

- Característica: Não mantém nem avalia uma política explícita durante as iterações. A política ótima só é extraída de forma gulosa ao final, quando $V(s)$ convergir.

Iteração de Política:

- Mecanismo: Mantém uma política explícita π e alterna entre duas etapas até a convergência:

1. Avaliação de Política: Calcula exatamente (ou aproxima) a função valor $V^\pi(s)$ para a política atual π , resolvendo um sistema de equações lineares:

$$E_\pi(s) = R(s) - C(\pi(s)) + \gamma \sum_{s'} P(s' | s, \pi(s)) E_\pi(s')$$

2. Melhoria de Política: Atualiza a política $\pi(s)$ de forma gulosa em relação aos valores $V^\pi(s)$ recém-calculados.
- Característica: Requer geralmente menos iterações que a Iteração de Valor para convergir, porém cada iteração é mais custosa computacionalmente devido à resolução do sistema de equações lineares.

Real-Time Dynamic Programming (RTDP):

- Mecanismo: É uma variante assíncrona da Iteração de Valor focada no planejamento em tempo real durante a execução ou simulação de episódios.
- Funcionamento: Em vez de realizar uma varredura completa atualizando todos os estados do ambiente (o que é inviável para espaços de estados imensos), o RTDP realiza atualizações de Bellman apenas nos estados visitados ao longo de trajetórias simuladas a partir de um estado inicial até o estado meta.
- Característica: Concentra os recursos computacionais exclusivamente nos estados que são relevantes e alcançáveis sob a política atual, sem a necessidade de processar todo o espaço de estados S .

Algoritmo	O que é atualizado	Varredura do Espaço de Estados
Iteração de Valor (IV)	Função valor $V(s)$	Varredura completa ($s \in S$)
Iteração de Política (IP)	Política $\pi(s)$ e Valor $V^\pi(s)$	Varredura completa ($s \in S$)
RTDP	Função valor $V(s)$	Amostrada/Focada em trajetórias relevantes

Exercício 9

Dê uma definição (informal) de Aprendizado por Reforço.

Resposta:

O Aprendizado por Reforço é um paradigma de aprendizado de máquina no qual um agente aprende a tomar decisões em um ambiente dinâmico por meio de tentativa e erro.

Diferente do aprendizado supervisionado, o agente não recebe instruções explícitas sobre qual ação tomar em cada situação; em vez disso, ele executa ações no ambiente e recebe retornos numéricos chamados de recompensas (positivas) ou punições (negativas).

O objetivo principal do agente é aprender uma política de comportamento que maximize a quantidade total de recompensa acumulada ao longo do tempo.

Principais Características:

- Aprendizado sem Supervisor: Não há um professor para rotular as ações corretas ou incorretas. O sinal de feedback indica apenas quão bom ou ruim foi o resultado de uma ação.
- Recompensa Atrasada: Uma ação tomada no presente pode afetar não apenas a recompensa imediata, mas também todas as recompensas futuras.
- Exploration vs. Exploitation: Para obter altas recompensas, o agente deve preferir ações que já testou no passado e se mostraram eficazes (*exploração*). No entanto, para descobrir tais ações, ele precisa experimentar opções nunca antes testadas (*exploração*).

Exercício 10

Explique a diferença entre os seguintes conceitos de Aprendizado por Reforço (AR):

- a) Predição Vs Controle
- b) *Model-Based* Vs *Model-Free*
- c) Aprendizado por Reforço *on-policy* Vs *off-policy*

Resposta:

a)

A distinção entre Predição e Controle aborda duas metas fundamentais no aprendizado por reforço:

- Predição (Avaliação de Política):
 - Determinar a função de valor $V^\pi(s)$ (ou $Q^\pi(s, a)$) para uma política π fixa e dada.
 - O problema responde à pergunta: "Quão boa é essa política específica?". O agente observa as transições e recompensas geradas ao seguir a política π e estima o retorno esperado para cada estado s , sem tentar modificar a política durante o processo.
- Controle (Otimização de Política):
 - Encontrar a política ótima π^* que maximiza o retorno acumulado esperado no ambiente.
 - O problema responde à pergunta: "Qual é a melhor forma de agir?". Envolve combinar a avaliação de política com a melhoria contínua da política (como no paradigma de *Generalized Policy Iteration*), buscando selecionar as ações que levam aos melhores resultados de longo prazo.

b)

Essa classificação diz respeito ao conhecimento ou aprendizado prévio do agente sobre as dinâmicas do ambiente:

- Model-Based
 - O agente possui ou aprende um modelo explícito do ambiente. Esse modelo consiste no modelo de transição $P(s' | s, a)$ (a probabilidade de ir para o estado s' ao tomar a ação a no estado s) e na função de recompensa $R(s, a)$.
 - Permite que o agente faça planejamento (como simular trajetórias hipotéticas antes de agir) e use técnicas de programação dinâmica. No entanto, aprender ou armazenar um modelo exato do ambiente pode ser extremamente custoso ou inviável em termos computacionais para espaços de estados grandes.
- Model-Free
 - O agente não possui e nem tenta aprender as probabilidades de transição $P(s' | s, a)$ ou o modelo $R(s, a)$.
 - Em vez de planejar com base no ambiente, o agente aprende diretamente funções de valor (como $Q(s, a)$ no *Q-Learning*) ou políticas ótimas a partir do feedback direto de suas interações reais (amostras). É ideal para ambientes complexos ou estocásticos onde modelar as regras exatas do mundo é inviável.

c)

Esta diferença refere-se ao relacionamento entre a política que o agente utiliza para gerar ações no ambiente e a política que ele está tentando avaliar ou otimizar:

- On-Policy
 - O agente aprende o valor da política que ele está de fato executando para explorar o ambiente.
 - A política de comportamento (usada para escolher ações, muitas vezes estocástica/exploratória como *ϵ -greedy*) é a mesma política de aprendizado que está sendo avaliada e melhorada.
 - Algoritmo *SARSA*, onde a atualização de $Q(s, a)$ leva em consideração a próxima ação a' que o agente realmente executará segundo a sua política atual.
- Off-Policy
 - O agente avalia ou melhora uma política (chamada de *política de aprendizado* ou *target policy*) enquanto executa ações geradas por uma política diferente (chamada de *política de comportamento* ou *behavior policy*).
 - Permite ao agente aprender uma política ótima (por exemplo, totalmente gulosa) enquanto continua explorando o ambiente por meio de uma política estocástica.
 - Algoritmo *Q-Learning*, onde a atualização do valor de um estado-ação $Q(s, a)$ assume que a próxima ação a' será a que maximiza o valor de Q no próximo estado ($\max_{a'} Q(s', a')$), independentemente de qual ação o agente venha de fato a escolher no passo seguinte.

Exercício 11

Considere o cálculo da média incremental (Slides 34–36 da Aula 19), também chamada de média móvel.

- a. Demonstre passo a passo a fórmula da média incremental μ_k para uma sequência de x_j valores.

- b. Mostre (explique) como a média incremental é aplicada no método de predição de Monte Carlo para RL. Neste caso, o que seria μ_k ? E o que seria x_j ?
- c. Mostre (explique) como a média incremental é aplicada no método de predição TD(0). Neste caso, o que seria μ_k ? E o que seria x_j ?
- d. Explique porque podemos dizer que "na média incremental corrigimos a média na direção do erro".

Resposta:

(a) Demonstração passo a passo da fórmula da média incremental

Objetivo: Demonstrar como a média de uma sequência de k valores $x_1, x_2, \dots, x_k$ pode ser computada de forma recursiva/incremental μ_k a partir da média anterior μ_{k-1} e do novo valor amostrado x_k .

Demonstração Passo a Passo. Por definição, a média aritmética μ_k de uma sequência de k amostras é dada por:

$$\mu_k = \frac{1}{k} \sum_{j=1}^k x_j \quad (1)$$

Isolando o último termo (x_k) da somatória, podemos reescrever:

$$\mu_k = \frac{1}{k} \left(\sum_{j=1}^{k-1} x_j + x_k \right) \quad (2)$$

Multiplicando e dividindo a soma dos primeiros $k-1$ termos por $(k-1)$, reconhecemos a definição da média anterior $\mu_{k-1} = \frac{1}{k-1} \sum_{j=1}^{k-1} x_j$:

$$\mu_k = \frac{1}{k} ((k-1)\mu_{k-1} + x_k) \quad (3)$$

Expandindo e distribuindo o fator $\frac{1}{k}$:

$$\mu_k = \frac{k-1}{k} \mu_{k-1} + \frac{1}{k} x_k = \left(1 - \frac{1}{k} \right) \mu_{k-1} + \frac{1}{k} x_k \quad (4)$$

Reagrupando os termos em função da média anterior μ_{k-1} :

$$\mu_k = \mu_{k-1} - \frac{1}{k} \mu_{k-1} + \frac{1}{k} x_k \quad (5)$$

Fatorando o termo $\frac{1}{k}$, obtemos a **fórmula da média incremental**:

$$\mu_k = \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1}) \quad (6)$$

□

De forma geral, em aprendizado de máquina, substitui-se o fator $\frac{1}{k}$ por uma taxa de aprendizado (*learning rate*) $\alpha \in (0, 1]$, resultando em:

$$\mu_k \leftarrow \mu_{k-1} + \alpha(x_k - \mu_{k-1}) \quad (7)$$

(b) Aplicação no método de predição Monte Carlo (MC) para RL

No método de Monte Carlo (*Model-Free Prediction*), o objetivo é estimar a função valor de estado $V^\pi(s)$ para uma política fixa π sem conhecer as probabilidades de transição T ou as recompensas R .

O aprendizado ocorre ao fim de episódios completos. A atualização incremental do valor de um estado s é expressa por:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha (G_{\text{total}}(s) - V^\pi(s)) \quad (8)$$

Mapeamento das variáveis:

- μ_k corresponde à estimativa atualizada da função valor do estado s , isto é, $V^\pi(s)$.
- μ_{k-1} corresponde à estimativa anterior da função valor antes da observação do episódio, $V^\pi(s)$.
- x_j (ou x_k) corresponde ao **retorno total acumulado observado** a partir da visita ao estado s até o final do episódio, denotado por $G_{\text{total}}(s)$ (também conhecido como *reward-to-go*):

$$G_{\text{total}}(s) = \sum_{t=0}^T \gamma^t R_{t+1} \quad (9)$$

- O fator de ponderação $\frac{1}{k}$ é substituído pelo parâmetro α (taxa de aprendizado).

(c) Aplicação no método de predição Temporal Difference $TD(0)$

Diferente de Monte Carlo, o método $TD(0)$ não aguarda o término do episódio completo para atualizar as estimativas. Ele realiza a atualização a cada passo/transição individual (s, a, s', r) no ambiente (*bootstrapping*).

A regra de atualização incremental do $TD(0)$ é dada por:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha (R(s) + \gamma V^\pi(s') - V^\pi(s)) \quad (10)$$

Mapeamento das variáveis:

- μ_k corresponde à nova estimativa da função valor do estado s , $V^\pi(s)$.
- μ_{k-1} corresponde à estimativa antiga de $V^\pi(s)$.
- x_j (ou x_k) corresponde ao **alvo temporal** (*TD Target*), denotado por $R(s) + \gamma V^\pi(s')$, que representa uma amostra ruidosa da recompensa imediata observada $R(s)$ somada ao valor descontado do estado sucessor s' .
- A diferença $(x_k - \mu_{k-1})$ equivale a $R(s) + \gamma V^\pi(s') - V^\pi(s)$, conhecida como o **Erro TD** (*TD Error*, δ_t).

(d) Explicação de "na média incremental corrigimos a média na direção do erro"

A estrutura geral da atualização incremental possui o seguinte padrão:

$$\text{NovoValor} \leftarrow \text{ValorAntigo} + \text{Passo} \times [\text{Alvo} - \text{ValorAntigo}] \quad (11)$$

O termo entre colchetes, **Erro** = (**Alvo** – **ValorAntigo**), mede a discrepância entre a estimativa atual e a observação realizada.

1. **Se Alvo > ValorAntigo (Erro > 0):** A estimativa atual subestimava a recompensa real do ambiente. O erro positivo faz com que o termo $\alpha \times \text{Erro}$ seja somado, **aumentando** o valor da média na direção do alvo obtido.
2. **Se Alvo < ValorAntigo (Erro < 0):** A estimativa atual superestimava a recompensa. O erro negativo subtrai valor da estimativa anterior, **reduzindo** a média em direção ao alvo real.

Portanto, em cada iteração, ajustamos a estimativa dando um "passo" proporcional a α **na direção exata em que o erro aponta**, eliminando a necessidade de armazenar todo o histórico de amostras do passado na memória ($O(1)$ em espaço e tempo).

Exercício 12

Em um ambiente que pode ser representado por um MDP, dada uma política π para esse ambiente, podemos estimar a função valor da política, $V^\pi(s)$, através de interações com o ambiente, mesmo quando desconhecemos o MDP. Complete as frases:

- a. _____ são métodos que estimam a função valor $V^\pi(s)$ amostrando episódios completos de interações com o ambiente.
- b. _____ são métodos que estimam a função valor $V^\pi(s)$ amostrando 1 passo a frente de interação com o ambiente e em seguida usam uma estimativa do restante do episódio.
- c. A atualização de $V^\pi(s)$ usando o método de Monte-Carlo para predição é dada por:

- d. A atualização de $V^\pi(s)$ usando o método de $TD(0)$ para predição é dada por:

Resposta:

Resolução Detalhada e Preenchimento das Lacunas

Com base no livro *Artificial Intelligence: A Modern Approach* (AIMA – Cap. 21) e nas formulações de Aprendizado por Reforço Passivo (*Passive Reinforcement Learning*):

(a) Métodos de Monte Carlo

Resposta para completar o texto: Métodos de Monte Carlo (ou *Aprendizado por Monte Carlo / Direct Utility Estimation*)

Explicação segundo o AIMA: No método de estimativa direta de utilidade (*Direct Utility Estimation / Monte Carlo*), o agente executa uma sequência de experimentos/episódios até atingir um estado terminal. A utilidade (ou valor) de um estado s é definida como o retorno acumulado esperado (*reward-to-go*) a partir do estado s . Como o ambiente não é conhecido a priori, a amostragem é feita gerando ****episódios completos****, e a estimativa da função valor $V^\pi(s)$ é calculada pela média dos retornos totais obtidos após passar por s .

(b) Métodos de Diferença Temporal (TD Learning)

Resposta para completar o texto: Métodos de Diferença Temporal (TD / Temporal Difference)

Explicação segundo o AIMA: Diferente do método de Monte Carlo, que aguarda o episódio finalizar para obter o retorno total real, os métodos de ****Diferença Temporal (TD)**** usam a técnica de *bootstrapping*. Eles observam a transição imediata de um único passo de interação $(s, \pi(s), s', r)$, recebem a recompensa r , e ****usam a estimativa atual do valor do estado sucessor $V^\pi(s')$ **** como uma aproximação para o restante do episódio.

(c) Regra de Atualização do Método de Monte Carlo

Resposta para completar a lacuna:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha (G_{\text{total}}(s) - V^\pi(s)) \quad (12)$$

Explicação dos termos (AIMA):

- $G_{\text{total}}(s) = \sum_{t=0}^T \gamma^t R_{t+1}$ é o retorno total descontado acumulado observado a partir da visita ao estado s até o término do episódio.
- $\alpha \in (0, 1]$ é a taxa de aprendizado (*learning rate*).
- $(G_{\text{total}}(s) - V^\pi(s))$ é a diferença entre a recompensa real observada no episódio e a estimativa atual do valor do estado.

(d) Regra de Atualização do Método TD(0)

Resposta para completar a lacuna:

$$V^\pi(s) \leftarrow V^\pi(s) + \alpha (R(s) + \gamma V^\pi(s') - V^\pi(s)) \quad (13)$$

Explicação dos termos (AIMA):

- $R(s)$ é a recompensa imediata observada na transição do estado s para s' .
- $\gamma \in [0, 1]$ é o fator de desconto (*discount factor*).
- $R(s) + \gamma V^\pi(s')$ é o ****alvo temporal**** (*TD Target*), formado pela recompensa observada no passo atual somada à utilidade estimada descontada do estado sucessor s' .
- $(R(s) + \gamma V^\pi(s') - V^\pi(s))$ é o ****Erro TD**** (δ_t), que mede o desvio em relação à Equação de Bellman para uma única transição.

Questão 13

Considerando a atualização da função valor usando os métodos MC e TD, explique brevemente os seguintes conceitos:

- “model-free RL”
- “total (episode) return”
- “TD target”

- d. “TD error”
- e. “bootstrapping”

Resposta:

(a) Model-Free RL (Aprendizado por Reforço Livre de Modelo)

O termo **Model-Free RL** refere-se a algoritmos que aprendem uma política ótima ou estimam a função valor de estado/ação $V(s)$ ou $Q(s, a)$ **sem estimar ou construir explicitamente o modelo transicional do ambiente** (isto é, sem estimar as probabilidades de transição de estado $P(s'|s, a)$ nem a função de recompensa $R(s, a)$).

- Os métodos Monte Carlo (MC) e Diferença Temporal (TD) são exemplos clássicos de abordagens *model-free*.
- A grande vantagem é a eficiência computacional por passo, pois evita a necessidade de resolver um sistema de equações completo ou calcular o somatório sobre todos os possíveis estados sucessores s' , utilizando apenas as transições observadas diretamente nas interações com o ambiente.

(b) Total (Episode) Return (Retorno Total do Episódio)

O **Retorno Total de um Episódio**, denotado por G_t ou $G_{\text{total}}(s)$, representa a soma de todas as recompensas acumuladas e descontadas obtidas a partir de um instante de tempo t (ou da visita a um estado s) até o término do episódio no estado terminal T :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \quad (14)$$

onde $\gamma \in [0, 1]$ é o fator de desconto. No método de **Monte Carlo**, o valor real G_t observado ao fim do episódio é utilizado como o valor-alvo (*target*) para atualizar a estimativa $V(s)$.

(c) TD Target (Alvo Temporal)

O **Alvo Temporal** (*TD Target*) é a estimativa do valor futuro utilizada pelo método $TD(0)$ para atualizar o valor do estado atual s_t . Ele é composto pela recompensa imediata observada R_{t+1} somada ao valor estimado do estado sucessor $V(s_{t+1})$, descontado por γ :

$$\text{TD Target} = R_{t+1} + \gamma V(s_{t+1}) \quad (15)$$

Enquanto no método Monte Carlo o alvo é o retorno real completo do episódio (G_t), no $TD(0)$ o alvo é uma amostra composta de apenas **1 passo de experiência real mais a estimativa atual do valor restante**.

(d) TD Error (Erro TD)

O **Erro TD** (denotado por δ_t) é a diferença/discrepância entre o alvo temporal estimado (*TD Target*) e a estimativa atual da função valor para o estado s_t :

$$\delta_t = \underbrace{R_{t+1} + \gamma V(s_{t+1})}_{\text{TD Target}} - \underbrace{V(s_t)}_{\text{Estimativa Antiga}} \quad (16)$$

O Erro TD quantifica o "quanto o resultado observado no passo imediato surpreendeu a estimativa anterior do agente". Na regra de atualização do $TD(0)$, o valor do estado é ajustado proporcionalmente a este erro: $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$.

(e) Bootstrapping

Em Aprendizado por Reforço, *Bootstrapping* refere-se à prática de atualizar uma estimativa da função valor com base em outra estimativa previamente existente, em vez de esperar por uma observação completa/real dos resultados até o fim do episódio.

- O método $TD(0)$ utiliza *bootstrapping* porque atualiza $V(s_t)$ utilizando a estimativa $V(s_{t+1})$ do estado sucessor.
- O método de Monte Carlo **não** realiza *bootstrapping*, pois aguarda o encerramento do episódio para utilizar o retorno acumulado real G_t .

Questão 14

O problema do macaco e das bananas é enfrentado por um macaco em um laboratório com algumas bananas penduradas no teto, fora de seu alcance. Há uma caixa disponível que permitirá ao macaco alcançar as bananas se ele subir nela. Inicialmente, o macaco está na localização A , as bananas em B e a caixa em C . O macaco e a caixa têm a altura *Baixa*, mas, se subir na caixa, o macaco terá a altura *Alta*, a mesma altura das bananas. As ações disponíveis para o macaco incluem *Ir* de um lugar para outro, *Empurrar* um objeto de um lugar para outro, *Subir* em um objeto ou *Descer* de um objeto, e ainda *Pegar* ou *Soltar* um objeto. O resultado de *Pegar* é que o macaco segura o objeto, se o macaco e o objeto estiverem no mesmo lugar e na mesma altura.

- Escreva a descrição do estado inicial.
- Escreva os seis operadores de ação.

Resposta:

A formalização a seguir adota a sintaxe padrão de planejamento no estilo STRIPS/PDDL conforme apresentada no livro AIMA (Capítulo 10).

Constantes e Predicados

Símbolos de Constantes:

- **Localizações:** A, B, C
- **Alturas:** *Baixa, Alta*

- **Objetos/Agentes:** *Macaco, Bananas, Caixa*

Predicados Utilizados:

- $At(x, l)$: O objeto/agente x está na localização l .
- $Height(x, h)$: O objeto/agente x está na altura h .
- $On(x, y)$: O objeto/agente x está em cima do objeto y .
- $OnFloor(x)$: O objeto/agente x está no chão.
- $Has(x, y)$: O agente x segura/possui o objeto y .
- $EmptyHand(x)$: O agente x está com as mãos livres.

(a) Descrição do Estado Inicial (*Init*)

O estado inicial especifica a posição e altura de cada entidade no ambiente no início do problema:

$$Init(At(Macaco, A) \wedge Height(Macaco, Baixa) \wedge OnFloor(Macaco) \wedge EmptyHand(Macaco) \wedge \\ At(Bananas, B) \wedge Height(Bananas, Alta) \wedge \\ At(Caixa, C) \wedge Height(Caixa, Baixa) \wedge OnFloor(Caixa))$$

(b) Descrição dos Seis Operadores de Ação

1. GoFromTo(*from, to*)

O macaco caminha de uma localização para outra no chão.

- **PRECOND:** $At(Macaco, from) \wedge OnFloor(Macaco) \wedge Height(Macaco, Baixa)$
- **EFFECT:** $At(Macaco, to) \wedge \neg At(Macaco, from)$

2. Push(*obj, from, to*)

O macaco empurra a caixa de uma localização para outra (ambos no chão e na mesma posição inicial).

- **PRECOND:** $At(Macaco, from) \wedge At(obj, from) \wedge OnFloor(Macaco) \wedge OnFloor(obj) \wedge Height(Macaco, Baixa)$
- **EFFECT:** $At(Macaco, to) \wedge At(obj, to) \wedge \neg At(Macaco, from) \wedge \neg At(obj, from)$

3. ClimbUp(*obj, loc*)

O macaco sobe em cima de um objeto (caixa) localizado na mesma posição, mudando sua altura para *Alta*.

- **PRECOND:** $At(Macaco, loc) \wedge At(obj, loc) \wedge OnFloor(Macaco) \wedge Height(Macaco, Baixa)$
- **EFFECT:** $On(Macaco, obj) \wedge Height(Macaco, Alta) \wedge \neg OnFloor(Macaco) \wedge \neg Height(Macaco, Baixa)$

4. ClimbDown(*obj, loc*)

O macaco desce do objeto para o chão, voltando à altura *Baixa*.

- **PRECOND:** $At(Macaco, loc) \wedge On(Macaco, obj) \wedge Height(Macaco, Alta)$
- **EFFECT:** $OnFloor(Macaco) \wedge Height(Macaco, Baixa) \wedge \neg On(Macaco, obj) \wedge \neg Height(Macaco, Alta)$

5. **Grab**(*obj, loc, h*)

O macaco pega um objeto se ambos estiverem na mesma localização e na mesma altura, e se a mão do macaco estiver livre.

- **PRECOND:** $At(Macaco, loc) \wedge At(obj, loc) \wedge Height(Macaco, h) \wedge Height(obj, h) \wedge EmptyHand(Macaco)$
- **EFFECT:** $Has(Macaco, obj) \wedge \neg EmptyHand(Macaco)$

6. **Drop**(*obj, loc, h*)

O macaco solta um objeto que está segurando.

- **PRECOND:** $Has(Macaco, obj) \wedge At(Macaco, loc) \wedge Height(Macaco, h)$
- **EFFECT:** $EmptyHand(Macaco) \wedge At(obj, loc) \wedge Height(obj, h) \wedge \neg Has(Macaco, obj)$

Questão 15

Digamos que você queira desenvolver um sistema de planejamento para um robô Limpador de Cozinhas.

Parte a. Escreva o conjunto de operadores STRIPS necessários para resolver esse problema considerando as seguintes propriedades do domínio:

- limpar o fogão ou a geladeira deixarão o chão sujo;
- o fogão deve ser limpo antes de fechar a tampa do fogão;
- limpar a geladeira gera lixo e suja a mesa da cozinha;
- limpar a mesa ou o chão suja a pia.

Sua resposta deve usar somente os seguintes predicados ou proposições: *limpo*(*x*) (sendo que $\neg \text{limpo}(x)$ denota que *x* está sujo), *lixeira-cheia*, *fogão-fechado* e *fogão-aberto*. Além disso, você deve usar os símbolos constantes: *Fogão*, *Chão*, *Geladeira*, *Mesa* e *Pia*. Lembre que os operadores STRIPS podem conter literais positivos e negativos, porém todas as pré-condições devem ser positivas.

Parte b. Escreva uma descrição do estado inicial de uma cozinha em que estão sujos: o fogão, a geladeira, a mesa e o chão. A pia está limpa e a lixeira está vazia.

Parte c. Escreva uma descrição do estado meta em que tudo está limpo, o fogão está fechado, não importando se a lixeira está cheia ou vazia.

Parte d. Qual seria um plano solução para esse problema? Use sua intuição.

Resposta:

(Parte a) Conjunto de Operadores STRIPS

Para respeitar a restrição de que ****todas as pré-condições devem ser positivas**** (sem o operador de negação $\neg$ nas pré-condições), expressamos a condição "estar vazio/não-

cheia” de forma afirmativa ou omitimos a verificação de lixeira onde ela é efeito, mantendo as pré-condições estritamente como literais positivos.

1. Limpar-Geladeira

Propriedade (i e iii): Limpar a geladeira deixa o chão sujo, gera lixo (lixeira-cheia) e suja a mesa.

- **PRECOND:** *fogão-aberto* (ou sem pré-condições especiais para início de limpeza)
- **EFFECT:** $\text{limpo}(\text{Geladeira}) \wedge \neg \text{limpo}(\text{Chão}) \wedge \neg \text{limpo}(\text{Mesa}) \wedge \text{lixeira-cheia}$

2. Limpar-Fogão

Propriedade (i e ii): O fogão deve ser limpo antes de fechar a tampa (requer fogão-aberto) e deixa o chão sujo.

- **PRECOND:** *fogão-aberto*
- **EFFECT:** $\text{limpo}(\text{Fogão}) \wedge \neg \text{limpo}(\text{Chão})$

3. Fechar-Fogão

Propriedade (ii): Fecha a tampa do fogão (requer que o fogão esteja limpo e aberto).

- **PRECOND:** $\text{limpo}(\text{Fogão}) \wedge \text{fogão-aberto}$
- **EFFECT:** $\text{fogão-fechado} \wedge \neg \text{fogão-aberto}$

4. Limpar-Mesa

Propriedade (iv): Limpar a mesa suja a pia.

- **PRECOND:** *fogão-aberto*
- **EFFECT:** $\text{limpo}(\text{Mesa}) \wedge \neg \text{limpo}(\text{Pia})$

5. Limpar-Chão

Propriedade (iv): Limpar o chão suja a pia.

- **PRECOND:** *fogão-aberto*
- **EFFECT:** $\text{limpo}(\text{Chão}) \wedge \neg \text{limpo}(\text{Pia})$

6. Limpar-Pia

Limpa a pia sem gerar sujeira colateral em outros aparelhos.

- **PRECOND:** *fogão-aberto*
- **EFFECT:** $\text{limpo}(\text{Pia})$

7. Esvaziar-Lixeira

Esvazia a lixeira quando estiver cheia.

- **PRECOND:** *lixeira-cheia*
- **EFFECT:** $\neg \text{lixeira-cheia}$

(Parte b) Descrição do Estado Inicial (*Init*)

Conforme o enunciado: o fogão, a geladeira, a mesa e o chão estão sujos (ausência de *limpo(x)*). A pia está limpa. A lixeira está vazia (ausência de *lixeira-cheia*). O fogão encontra-se inicialmente aberto.

$$Init(limpo(Pia) \wedge fogão-aberto) \quad (17)$$

(Parte c) Descrição do Estado Meta (*Goal*)

A meta exige que todos os aparelhos/superfícies estejam limpos e que o fogão esteja fechado (o estado da lixeira é irrelevante):

$$Goal(limpo(Fogão) \wedge limpo(Geladeira) \wedge limpo(Mesa) \wedge limpo(Chão) \wedge limpo(Pia) \wedge fogão-fechado) \quad (18)$$

(Parte d) Plano Solução (Análise Causal e Sequência)

Análise das Dependências de Ordem Causal:

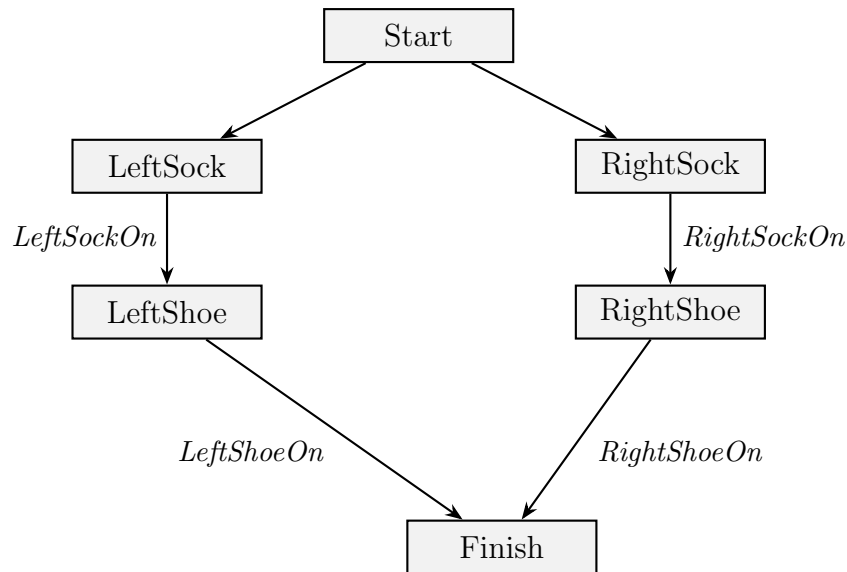
1. **Geladeira e Fogão primeiro:** Limpar a geladeira ou o fogão suja o chão e a mesa. Portanto, a geladeira e o fogão devem ser limpos antes da mesa e do chão.
2. **Mesa antes do Chão/Pia:** Limpar a geladeira suja a mesa. Limpar a mesa suja a pia.
3. **Chão antes da Pia:** Limpar o chão suja a pia. Logo, o chão deve ser limpo antes de limpar a pia.
4. **Pia por último:** Como limpar a mesa e o chão sujam a pia, a limpeza da pia deve ser a última ação de limpeza de superfícies.
5. **Fechar-Fogão:** Pode ser feito logo após limpar o fogão (enquanto o fogão estiver aberto e limpo).

Sequência de Ações do Plano Solução:

1. Limpar-Geladeira (*suja chão, mesa e enche lixeira*)
2. Limpar-Fogão (*limpa o fogão, suja chão*)
3. Fechar-Fogão (*fecha o fogão limpo*)
4. Limpar-Mesa (*limpa a mesa, suja a pia*)
5. Limpar-Chão (*limpa o chão, suja a pia*)
6. Limpar-Pia (*limpa a pia, concluindo que tudo está limpo*)

Questão 16

Considere o exemplo do livro AIMA (Planejamento de Ordem Parcial – POP), ilustrado pela figura abaixo:



Parte 1. Faça uma descrição dos operadores:

- *LeftSock* (colocar a meia no pé esquerdo)
- *LeftShoe* (colocar o sapato no pé esquerdo)
- *RightSock* (colocar a meia no pé direito)
- *RightShoe* (colocar o sapato no pé direito)

Parte 2. Mostre passo a passo a construção do plano acima utilizando o algoritmo POP.

Resposta:

(Parte 1) Descrição Formal dos Operadores STRIPS/PDDL

Para calçar o sapato é pré-requisito necessário que a meia correspondente já esteja calçada no mesmo pé. O estado inicial assume que ambos os pés estão sem meias e sem sapatos.

1. Action: LeftSock

- **PRECOND:** $BareFoot(Left)$ (pé esquerdo sem meia)
- **EFFECT:** $LeftSockOn \wedge \neg BareFoot(Left)$

2. Action: LeftShoe

- **PRECOND:** $LeftSockOn$
- **EFFECT:** $LeftShoeOn$

3. Action: RightSock

- **PRECOND:** $BareFoot(Right)$ (pé direito sem meia)
- **EFFECT:** $RightSockOn \wedge \neg BareFoot(Right)$

4. Action: RightShoe

- **PRECOND:** $RightSockOn$
- **EFFECT:** $RightShoeOn$

(Parte 2) Construção Passo a Passo pelo Algoritmo POP

O algoritmo de ****Planejamento de Ordem Parcial (POP)**** opera regressivamente a partir do objetivo (*Goal Regression*), adicionando ações para satisfazer precondições em aberto (*open preconditions*) e garantindo que não haja ameaças de ordem causal.

Passo 0: Inicialização do Plano Incompleto

- **Ação Inicial** (*Start*): Efeitos = $\{BareFoot(Left), BareFoot(Right)\}$
- **Ação Final** (*Finish*): Precondições = $\{LeftShoeOn, RightShoeOn\}$
- **Ordenação Inicial**: $Start < Finish$
- **Precondições em Aberto**: $(LeftShoeOn, Finish), (RightShoeOn, Finish)$

Passo 1: Resolver a Precondição em Aberto (*LeftShoeOn, Finish*)

- Seleciona a ação *LeftShoe*, cujos efeitos satisfazem *LeftShoeOn*.
- Adiciona o elo causal (*Causal Link*): $LeftShoe \xrightarrow{LeftShoeOn} Finish$.
- Adiciona as ordenações necessárias: $Start < LeftShoe < Finish$.
- ****Nova precondição em aberto criada:**** $(LeftSockOn, LeftShoe)$.

Passo 2: Resolver a Precondição em Aberto (*RightShoeOn, Finish*)

- Seleciona a ação *RightShoe*, cujos efeitos satisfazem *RightShoeOn*.
- Adiciona o elo causal: $RightShoe \xrightarrow{RightShoeOn} Finish$.
- Adiciona as ordenações: $Start < RightShoe < Finish$.
- ****Nova precondição em aberto criada:**** $(RightSockOn, RightShoe)$.

Passo 3: Resolver a Precondição em Aberto (*LeftSockOn, LeftShoe*)

- Seleciona a ação *LeftSock*, que produz o efeito *LeftSockOn*.
- Adiciona o elo causal: $LeftSock \xrightarrow{LeftSockOn} LeftShoe$.
- Conecta a precondição de *LeftSock* (*BareFoot(Left)*) diretamente ao estado *Start*:
 $Start \xrightarrow{BareFoot(Left)} LeftSock$.

Passo 4: Resolver a Precondição em Aberto (*RightSockOn, RightShoe*)

- Seleciona a ação *RightSock*, que produz o efeito *RightSockOn*.
- Adiciona o elo causal: $RightSock \xrightarrow{RightSockOn} RightShoe$.
- Conecta a precondição de *RightSock* (*BareFoot(Right)*) diretamente ao estado *Start*:
 $Start \xrightarrow{BareFoot(Right)} RightSock$.

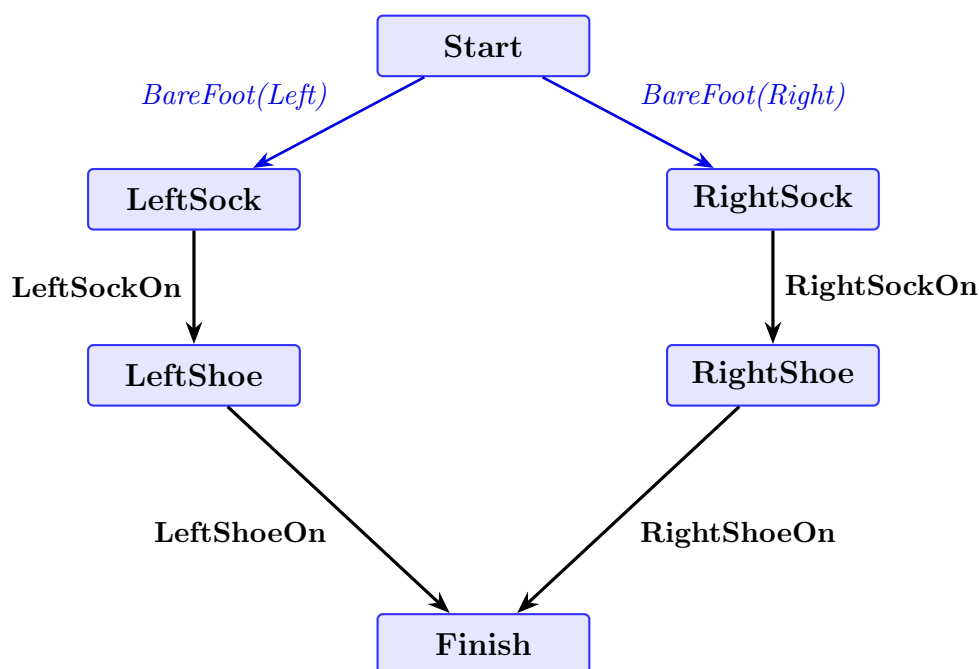
Passo 5: Verificação de Ameaças (*Threat Checking*)

- O algoritmo verifica se alguma ação ameaça destruir os elos causais estabelecidos.

- Como as cadeias para o pé esquerdo ($LeftSock \rightarrow LeftShoe$) e para o pé direito ($RightSock \rightarrow RightShoe$) utilizam recursos totalmente independentes, ****não existem conflitos nem ameaças**** entre os dois ramos.
- As ações dos dois lados podem ser intercaladas em qualquer ordem válida no plano final (ex: $LeftSock \rightarrow RightSock \rightarrow LeftShoe \rightarrow RightShoe$ ou vice-versa).

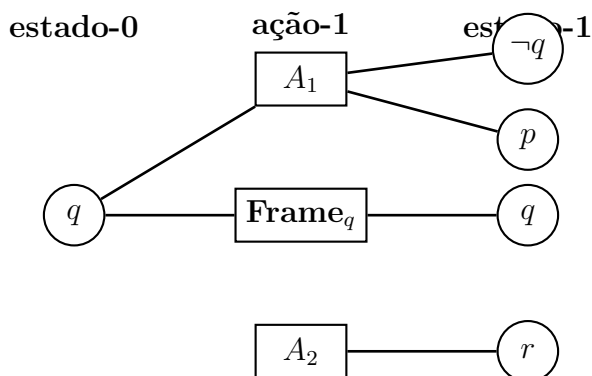
Grafo da Solução Final (Grafo POP)

A estrutura gráfica resultante do plano de ordem parcial construído é ilustrada no grafo a seguir:



Questão 17

Considere o problema de planejamento com 2 ações, A_1 e A_2 . A ação A_1 tem précondição q e efeitos $\{p, \neg q\}$. A ação A_2 não tem précondições e tem efeito r . O estado inicial é $\{q\}$. A meta é $\{p, r\}$. O grafo de planejamento expandido (gerado pelo algoritmo GRAPHPLAN) até o nível 1, sem marcações de mutexes, é dado a seguir:



PARTE 1: Marque os mutexes no grafo de planejamento (Nível 1).

PARTE 2: Estenda o grafo até o segundo nível (níveis ação-2 e estado-2) e complete a marcação dos demais mutexes.

PARTE 3: Mostre um plano válido (subgrafo) que corresponda a uma solução.

Resposta:

(PARTE 1) Marcação de Mutexes no Nível 1

Analisando as regras de exclusão mútua do Graphplan para a estrutura do enunciado:

Mutexes entre Ações (Nível Ação-1):

1. $\text{Mutex}(A_1, \text{Frame}_q)$: **Interferência / Efeitos Inconsistentes.** A_1 produz $\neg q$, destruindo o efeito e a precondição q mantidos pela persistência Frame_q .
2. $\text{Mutex}(A_1, A_2)$: **NÃO são mutex!** A_1 exige q e gera $\{p, \neg q\}$, enquanto A_2 não tem precondição e gera r . Nenhuma ação interfere na precondição ou efeito da outra.
3. Frame_q e A_2 : **NÃO são mutex.**

Mutexes entre Literais (Nível Estado-1):

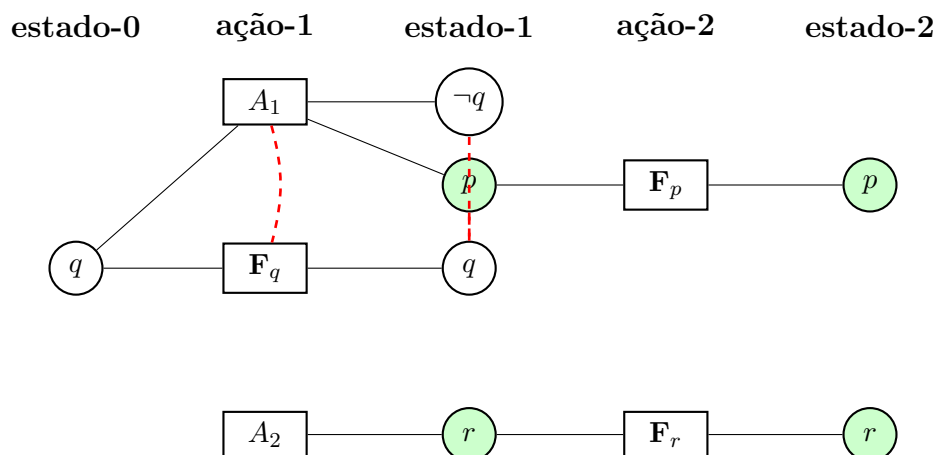
1. $\text{Mutex}(q, \neg q)$: **Literais Complementares.**
2. $\text{Mutex}(p, q)$: **Suportes Inconsistentes.** A única ação que produz p é A_1 , e a única ação que produz q é Frame_q . Como $\text{Mutex}(A_1, \text{Frame}_q)$, p e q são mutex.
3. $\text{Mutex}(p, r)$: **NÃO SÃO MUTEX!** Como A_1 (que produz p) e A_2 (que produz r) NÃO são mutex no nível ação-1, os literais p e r já podem ser obtidos juntos sem conflito no **estado-1**!

—

(PARTE 2) Expansão do Grafo para o Nível 2

Como as ações A_1 e A_2 são independentes e não-conflitantes no nível ação-1, o conjunto meta $\{p, r\}$ já se torna **livre de mutex no estado-1**.

Grafo de Planejamento Expandido até o Nível 2 com Marcações (TikZ):



—

(PARTE 3) Busca Regressiva e Subgrafo Solução

Realizando a busca de plano (*Backward Search*) no estado-1 a partir da meta $\{p, r\}$:

1. No **nível ação-1**, selecionamos: * Ação A_1 para gerar p (requer a condição q no estado-0). * Ação A_2 para gerar r (não requer condições). 2. Como A_1 e A_2 não possuem conflito de mutex entre si, ambas podem ser executadas **em paralelo no mesmo passo temporal (nível 1)**.

Plano Solução Extraído:

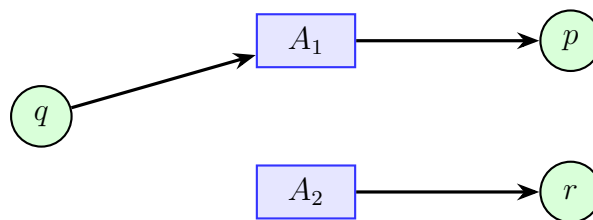
O plano paralelo obtido no primeiro passo é:

$$\text{Plano Solução} = [\{A_1, A_2\}] \quad (19)$$

Se for exigido um plano estritamente sequencial (uma ação por vez), qualquer ordem de execução entre A_1 e A_2 é válida:

$$\text{Plano Sequencial} = \langle A_1, A_2 \rangle \quad \text{ou} \quad \langle A_2, A_1 \rangle \quad (20)$$

Subgrafo Solução (TikZ):



Questão 18

Construa os níveis 0, 1 e 2 do grafo de planejamento para o problema da Figura 10.1 (Transporte de Carga Aérea).

Definição PDDL do Problema (Figura 10.1):

- **Init:** $\text{At}(C_1, \text{SFO}) \wedge \text{At}(C_2, \text{JFK}) \wedge \text{At}(P_1, \text{SFO}) \wedge \text{At}(P_2, \text{JFK}) \wedge \text{Cargo}(C_1) \wedge \text{Cargo}(C_2) \wedge \text{Plane}(P_1) \wedge \text{Plane}(P_2) \wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO})$
- **Goal:** $\text{At}(C_1, \text{JFK}) \wedge \text{At}(C_2, \text{SFO})$
- **Action** Load(c, p, a):
 - **PRECOND:** $\text{At}(c, a) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$
 - **EFFECT:** $\neg \text{At}(c, a) \wedge \text{In}(c, p)$
- **Action** Unload(c, p, a):
 - **PRECOND:** $\text{In}(c, p) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$
 - **EFFECT:** $\text{At}(c, a) \wedge \neg \text{In}(c, p)$
- **Action** Fly($p, \text{from}, \text{to}$):
 - **PRECOND:** $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$
 - **EFFECT:** $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$

Resposta:

No Grafo de Planejamento (*Planning Graph*), omitimos os predicados estáticos de tipo (como Cargo, Plane, Airport) para focar nos fluentes de estado (At e In).

Nível 0: Estado Inicial (S_0)

O conjunto de literais fluentes presentes no nível de estado S_0 consiste em:

$$S_0 = \{\text{At}(C_1, \text{SFO}), \text{At}(P_1, \text{SFO}), \text{At}(C_2, \text{JFK}), \text{At}(P_2, \text{JFK})\} \quad (21)$$

Nota: Não há relações de mutex no estado S_0 .

Nível 1: Ações (A_1) e Estados (S_1)

Ações Aplicáveis no Nível A_1 :

Com base nas precondições satisfeitas em S_0 :

1. **Carregamentos de Carga:**
 - Load(C_1, P_1, SFO) (produz In(C_1, P_1) e $\neg \text{At}(C_1, \text{SFO})$)
 - Load(C_2, P_2, JFK) (produz In(C_2, P_2) e $\neg \text{At}(C_2, \text{JFK})$)
2. **Voos de Aviões:**
 - Fly($P_1, \text{SFO}, \text{JFK}$) (produz At(P_1, JFK) e $\neg \text{At}(P_1, \text{SFO})$)
 - Fly($P_2, \text{JFK}, \text{SFO}$) (produz At(P_2, SFO) e $\neg \text{At}(P_2, \text{JFK})$)
3. **Ações de Persistência (Frame):**
 - Frame_{At}(C_1, SFO), Frame_{At}(P_1, SFO), Frame_{At}(C_2, JFK), Frame_{At}(P_2, JFK)

Principais Mutexes no Nível Ação A_1 :

- Mutex(Load(C_1, P_1, SFO), Fly($P_1, \text{SFO}, \text{JFK}$)): **Interferência / Efeitos Inconsistentes** (Fly remove a precondição/efeito At(P_1, SFO)).
- Mutex(Load(C_1, P_1, SFO), Frame_{At}(C_1, SFO)): **Efeitos Inconsistentes** (Load apaga At(C_1, SFO)).
- Mutex(Fly($P_1, \text{SFO}, \text{JFK}$), Frame_{At}(P_1, SFO)): **Efeitos Inconsistentes**.

Conjunto de Literais no Estado S_1 :

$$S_1 = \{\text{At}(C_1, \text{SFO}), \text{At}(P_1, \text{SFO}), \text{At}(C_2, \text{JFK}), \text{At}(P_2, \text{JFK}), \\ \text{In}(C_1, P_1), \text{In}(C_2, P_2), \text{At}(P_1, \text{JFK}), \text{At}(P_2, \text{SFO}), \neg \text{At}(C_1, \text{SFO}), \dots\}$$

Nível 2: Ações (A_2) e Estados (S_2)

No nível A_2 , surgem as ações de descarregamento (Unload), pois a precondição In(c, p) agora existe em S_1 .

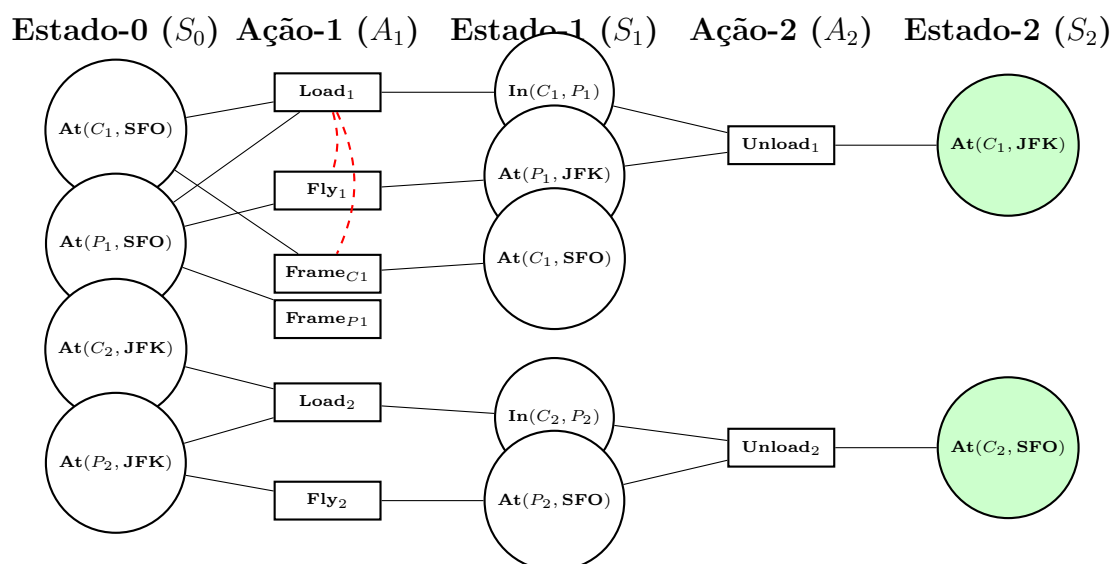
Novas Ações Aplicáveis no Nível A_2 :

- $\text{Unload}(C_1, P_1, \text{JFK})$ (requer $\text{In}(C_1, P_1)$ e $\text{At}(P_1, \text{JFK}) \rightarrow$ produz $\text{At}(C_1, \text{JFK})$)
- $\text{Unload}(C_2, P_2, \text{SFO})$ (requer $\text{In}(C_2, P_2)$ e $\text{At}(P_2, \text{SFO}) \rightarrow$ produz $\text{At}(C_2, \text{SFO})$)
- Diversas novas ações de voo e carregamento em novos locais, além das ações de persistência.

Saturação da Meta em S_2 :

No nível de estado S_2 , os literais da meta $\{\text{At}(C_1, \text{JFK}), \text{At}(C_2, \text{SFO})\}$ aparecem produzidos por $\text{Unload}(C_1, P_1, \text{JFK})$ e $\text{Unload}(C_2, P_2, \text{SFO})$. Como essas duas ações pertencem a subredes independentes (P_1/C_1 vs P_2/C_2), elas ****não são mutex****, permitindo alcançar a meta!

Grafo de Planejamento Expandido (Níveis 0, 1 e 2 em TikZ)



Questão 19

A Figura 10.4 mostra um problema de mundo de blocos conhecido como **Anomalia de Sussman**. O problema foi considerado anômalo porque os planejadores sem intercalações (*non-interleaved planners*) do início da década de 1970 não conseguiram resolvê-lo.

Estado Inicial (Start) Estado Meta (Goal)



Escreva uma definição do problema e resolva-o manualmente com o **Graphplan**. Um planejador sem intercalações é um planejador que, ao receber dois subobjetivos G_1 e G_2 , produz um plano para G_1 concatenado com um plano para G_2 ou vice-versa. Explique por que um planejador sem intercalações não consegue resolver esse problema.

Resposta:**1. Definição Formal do Problema (STRIPS / PDDL)****Estado Inicial (*Init*):**

$$Init = \{On(C, A), OnTable(A), OnTable(B), Clear(C), Clear(B), ArmEmpty\} \quad (22)$$

Estado Meta (*Goal*):

$$Goal = \{On(A, B), On(B, C)\} \quad (23)$$

2. Por que Planejadores Sem Intercalação Falham?

Um planejador sem intercalação (*non-interleaved planner*, como o *STRIPS original*) tenta resolver a meta decompondo-a em subobjetivos independentes: $G_1 = On(A, B)$ e $G_2 = On(B, C)$, exigindo resolver completamente G_1 e depois G_2 (ou vice-versa):

1. **Tentar resolver $G_1 = On(A, B)$ primeiro:** Para colocar A sobre B , é necessário tirar C de cima de A (colocando C na mesa) e então mover A para cima de B . *Resultado parcial:* A fica em cima de B . Mas para depois resolver $G_2 = On(B, C)$, B precisa ser movido para cima de C , o que exige **desfazer** a conquista prévia de $On(A, B)$ (tirando A de cima de B). O planejador entra em *loop* ou falha.
2. **Tentar resolver $G_2 = On(B, C)$ primeiro:** Para colocar B em cima de C , move-se B para C . *Resultado parcial:* B fica sobre C . Mas para depois resolver $G_1 = On(A, B)$, A precisa ser colocado sobre B , mas A ainda está preso embaixo de C ! Para tirar C de cima de A , B terá que ser tirado do topo de C , desfazendo G_2 .

Conclusão: A resolução requer **intercalar** passos das duas metas: liberar A primeiro (tirando C), depois resolver $G_2 = On(B, C)$, e por fim resolver $G_1 = On(A, B)$.

3. Resolução Passo a Passo com o Graphplan**Nível 0 (S_0):**

$$S_0 = \{On(C, A), OnTable(A), OnTable(B), Clear(C), Clear(B)\}$$

Nível 1 ($A_1 \rightarrow S_1$):Ações aplicáveis em S_0 :

- $Unstack(C, A) \rightarrow$ produz $Holding(C), Clear(A)$.
- Ações de Persistência (Frame).

Nível 2 ($A_2 \rightarrow S_2$):Com $Holding(C)$ e $Clear(A)$ disponíveis em S_1 :

- $PutDown(C) \rightarrow$ produz $OnTable(C), Clear(C), ArmEmpty$.

Nível 3 ($A_3 \rightarrow S_3$):

Com ArmEmpty , $\text{Clear}(B)$ e $\text{Clear}(C)$ em S_2 :

- $\text{PickUp}(B) \rightarrow$ produz $\text{Holding}(B)$.
- $\text{Stack}(B, C) \rightarrow$ (no próximo passo) produz $\text{On}(B, C)$.

Nível 4 ($A_4 \rightarrow S_4$):

Com $\text{On}(B, C)$, $\text{Clear}(A)$ e ArmEmpty :

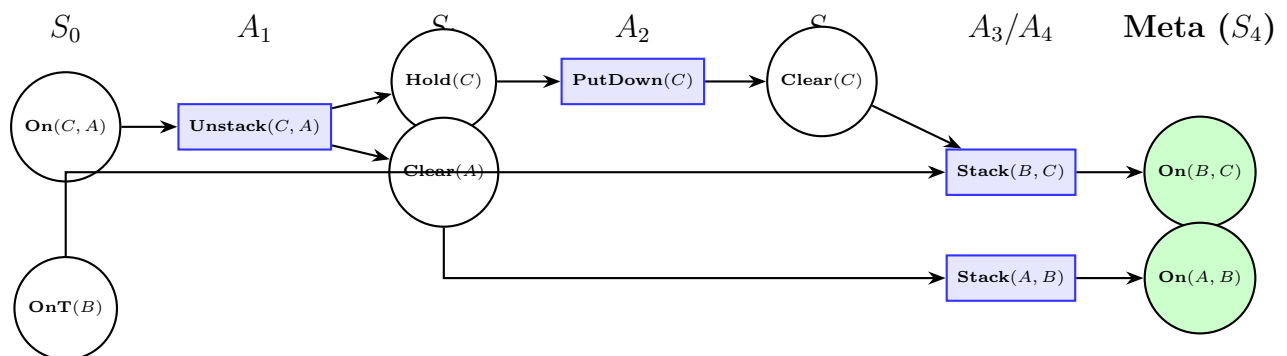
- $\text{PickUp}(A) \rightarrow$ pega A da mesa.
- $\text{Stack}(A, B) \rightarrow$ coloca A em cima de B .

No estado S_4 , a meta $\{\text{On}(A, B), \text{On}(B, C)\}$ é alcançada livre de mutexes!

—

4. Grafo de Planejamento da Solução (TikZ)

O subgrafo solução extraído do Graphplan representa o plano correto de 3 passos de movimentação (além de desempilhar):

**Plano Solução Extraído:**

$$\text{Plano} = \langle \text{Unstack}(C, A), \text{PutDown}(C), \text{Move}(B, \text{Table}, C), \text{Move}(A, \text{Table}, B) \rangle \quad (24)$$

Questão 20

PARTE A: Explique a ideia geral de extração automática de heurísticas de planejamento que fazem a relaxação dos efeitos negativos (Algoritmo do tipo Bellman–Ford, Aula 16, slides 16 e 22).

PARTE B: Explique as suposições feitas no cálculo das heurísticas $h_{\text{add}}^+(s)$ e $h_{\text{max}}^+(s)$ usando o algoritmo.

Resposta:**PARTE A: Idéia Geral de Extração Automática de Heurísticas por Relaxação dos Efeitos Negativos (P^+)****1. O Conceito da Relaxação P^+**

Na busca em planejamento clássico, calcular o custo exato do caminho mínimo $c^*(P)$ a partir de um estado s até a meta g é um problema intratável (PSPACE-completo). Para extrair funções heurísticas automaticamente a partir da própria descrição PDDL do problema, aplica-se a **relaxação da remoção de deletados (P^+)**:

- Ignoram-se os efeitos negativos das ações (isto é, considera-se apenas a lista de efeitos positivos $effects^+(a)$).
- **Monotonicidade:** Uma vez que um átomo p se torna verdadeiro em um estado, ele permanece verdadeiro para sempre.
- Se $c^*(P^+)$ é o custo ótimo do problema relaxado P^+ , a heurística perfeita relaxada é $h^+(s) = c^*(P^+)$. Como P^+ é uma relaxação válida do problema original P , $h^+(s)$ é **admissível**.

2. Aproximação Polinomial via Programação Dinâmica (Algoritmo Bellman–Ford)

Como calcular $h^+(s)$ de forma ótima ainda é NP-difícil, os algoritmos aproximam $h^+(s)$ atualizando iterativamente o custo estimado $h(p, s)$ para alcançar cada átomo $p \in \mathcal{P}$ a partir de s através de um algoritmo de **ponto-fixa (Bellman–Ford)**:

1. Inicialização:

$$h(p, s) = \begin{cases} 0, & \text{se } p \in s \\ \infty, & \text{caso contrário} \end{cases} \quad (25)$$

- 2. Propagação do Ponto-Fixo (Slide 16 e Slide 22):** Repete-se a atualização para cada ação aplicável a ($precond(a) \subseteq X$) e para cada $p \in effects^+(a)$ até que nenhum valor de $h(p, s)$ mude:

$$h(p, s) \leftarrow \min \{h(p, s), 1 + \text{Cost}(precond(a), s)\} \quad (26)$$

onde $\text{Cost}(precond(a), s)$ varia conforme a função heurística adotada (h_{add}^+ ou h_{max}^+).

PARTE B: Suposições e Análise de $h_{\text{add}}^+(s)$ e $h_{\text{max}}^+(s)$

As heurísticas $h_{\text{add}}^+(s)$ e $h_{\text{max}}^+(s)$ diferem na maneira como agregam os custos de múltiplos pré-requisitos para acionar uma ação e para satisfazer o conjunto final de metas $g = \{g_1, g_2, \dots, g_n\}$.

1. Heurística Aditiva: $h_{\text{add}}^+(s)$ (Slide 14 e 16)

- **Suposição de Independência de Submetas:** Assume que o custo para alcançar um conjunto de proposições C é a **soma** dos custos individuais para alcançar

cada proposição em C :

$$\text{Cost}_{\text{add}}(C, s) = \sum_{q \in C} h(q, s) \quad (27)$$

- **Regra de Atualização no Ponto-Fixo:**

$$h(p, s) \leftarrow \min \left\{ h(p, s), 1 + \sum_{q \in \text{precond}(a)} h(q, s) \right\} \quad (28)$$

- **Custo Final da Meta:**

$$h_{\text{add}}^+(s) = h(g, s) = \sum_{p \in g} h(p, s) \quad (29)$$

- **Propriedade e Qualidade (Slide 19):**

- ****NÃO é Admissível:**** Superestima o custo real em problemas onde ações sobrepostas/compartilhadas geram submetas simultaneamente (pois conta a mesma ação múltiplas vezes para alcançar submetas diferentes).
- ****Desempenho:**** É muito rápida e ****extremamente informativa**** para guiar a busca em problemas difíceis.

2. Heurística Máxima: $h_{\text{max}}^+(s)$ (Slide 20 e 22)

- **Suposição de Paralelismo / Custo Crítico:** Assume que o custo para alcançar um conjunto de proposições C é dominado estritamente pela submeta de ****maior custo**** (caminho crítico), assumindo paralelismo irrestrito no grafo:

$$\text{Cost}_{\text{max}}(C, s) = \max_{q \in C} h(q, s) \quad (30)$$

- **Regra de Atualização no Ponto-Fixo:**

$$h(p, s) \leftarrow \min \left\{ h(p, s), 1 + \max_{q \in \text{precond}(a)} h(q, s) \right\} \quad (31)$$

- **Custo Final da Meta:**

$$h_{\text{max}}^+(s) = h(g, s) = \max_{p \in g} h(p, s) \quad (32)$$

- **Propriedade e Qualidade (Slide 21):**

- ****É Admissível:**** Nunca superestima o custo do plano relaxado ótimo $h^+(s)$, pois $h_{\text{max}}^+(s) \leq h^+(s) \leq c^*(P)$.
- ****Desempenho:**** É ****pouco informativa**** (muito fraca), pois ignora o custo acumulado de ter que executar múltiplas ações para atingir submetas independentes.

—

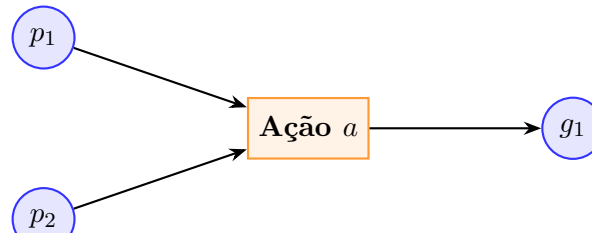
Tabela Comparativa Resumida (Slides 16 e 22)

Característica	Heurística $h_{\text{add}}^+(s)$	Heurística $h_{\text{max}}^+(s)$
Agregação em $\text{precond}(a)$	$\sum_{q \in \text{precond}(a)} h(q, s)$	$\max_{q \in \text{precond}(a)} h(q, s)$
Agregação na Meta g	$\sum_{p \in g} h(p, s)$	$\max_{p \in g} h(p, s)$
Suposição de Submetas	Totalmente Independentes	Paralelismo / Gargalo Máximo
Admissibilidade	Não (pode superestimar)	Sim (sempre subestima)
Informatividade	Alta (Guia bem a busca)	Baixa (Muitos empates)

Ilustração Gráfica da Propagação de Custo (TikZ)

O gráfico abaixo ilustra a diferença de cálculo entre h_{add}^+ e h_{max}^+ ao propagar os custos de um estado inicial $s = \{p_1, p_2\}$ através de uma ação a com $\text{precond}(a) = \{p_1, p_2\}$ e $\text{effects}^+(a) = \{g_1\}$:

Custos em s : $h(p_1) = 0, h(p_2) = 0$



Cálculo do Custo de g_1 para esta ação:

- **Via h_{add}^+ :** $h(g_1) = 1 + h(p_1) + h(p_2) = 1 + 0 + 0 = 1$.
 - **Via h_{max}^+ :** $h(g_1) = 1 + \max(h(p_1), h(p_2)) = 1 + \max(0, 0) = 1$.
- (Se $h(p_1) = 2$ e $h(p_2) = 3$, então $h_{\text{add}}^+(g_1) = 1 + 2 + 3 = 6$, enquanto $h_{\text{max}}^+(g_1) = 1 + \max(2, 3) = 4$).